# HEARTY OBP scaffold_1 QC workflow

See github for HEARTY: https://github.com/BiodiversityExtinction/HEARTY/tree/main

***Goal of HEARTY:*** *evaluate how sensitive heterozygosity and ROH inference are to different allele balance thresholds, particularly in low-coverage and historical samples.*

This workflow runs HEARTY on OBP BAM files to examine:

- minor allele balance distributions
- transition/transversion patterns
- window heterozygosity
- ROH sensitivity across different heterozygote thresholds.

## Why HEARTY is useful

### Problem

With low coveraege and historical data we are dealing with two conflicting issues: 1. Overestimation of het sites due to low coverage 2. Underdetection of het sites because of reference bias

### 1. Overestimation of heterozygous sites due to sequencing errors and damage

Low-coverage and historical genomic data can contain elevated levels of:

- sequencing error
- ancient DNA damage
- contamination

These issues can produce low-frequency alternate reads that may appear heterozygous even though they might be artefacts.

Therefore distinguishing true alternate alleles is tricky. If we have 20% of reads in a site that support the minor allele, could this be a true heterzygoste or a an error?

> Example high coverage site: Major allele count = 17 Minor allele count = 4 AB = 4 / 20 = 0.20

This becomes especially important in low-coverage data, where very small differences in read counts can strongly affect heterozygote inference:

> Example low coverage site: Major allele count = 5 Minor allele count = 1 AB = 1 / 5 = 0.20

Although both sites have the same allele balance, the lower-coverage site is much more likely to represent a false-positive heterozygous call caused by random sequencing noise or damage. This issue is particularly relevant when using genotype likelihood frameworks like ANGSD, where low-frequency minor reads contribute probabilistically to heterozygote likelihoods (see low coverage example above). Under low-coverage conditions, this uncertainty can sometimes inflate heterozygosity estimates relative to stricter genotype-calling approaches. In contrast, traditional genotype callers generally apply stricter genotype

assignment thresholds, which can reduce false-positive heterozygous calls but may also miss true variation in low-coverage samples.

### 2. Underdetection of heterozygous sites due to reference bias and low coverage uncertainty

Historical samples may also experience stronger reference bias because reads can be more diverged from the (often) modern reference genome. This can lead to:

- reduced mapping of alternate alleles
- preferential alignment of reference-supporting reads
- excess homozygous reference calls

As a result, true heterozygous sites may be missed, leading to underestimation of heterozygosity and genetic diversity.

### Consequence

Strict thresholds might remove true biological variation but lenient thresholds might lead to false positives.

### How HEARTY is helpful

HEARTY is useful because i allows these competing effects to be explored systematically across multiple allele balance thresholds and filtering schemes.

## Main question HEARTY addresses

***"How much do heterozygosity and ROH results change depending on how strict I am about calling heterozygous sites and defining ROHs?"***

A key robustness test is: ***"Do the biological conclusions remain consistent under different or stricter thresholds?"***

## Main goal of HEARTY

The main purpose of HEARTY is to determine biologically reasonable minor allele balance thresholds for a dataset by systematically comparing:

- heterozygosity estimates
- transition/transversion patterns
- window heterozygosity
- ROH inference

across multiple filtering thresholds.

Rather than assuming a single threshold is correct, HEARTY allows sensitivity analyses to determine whether conclusions are robust to parameter choice.

## Main vairables that HEARTY examines

1. Minor AB threshold: 0.15, 0.20, 0.35...

2. Site set: Transitions+Transversion, Transversions only
3. Alpha: 0.2, 0.3, 0.5

## Metrics

- mean heterozygosity
- raw FROH
- bridged FROH
- FROH >1 Mb
- FROH >5 Mb
-

## Inputs and requierements

- BAM list: one BAM per line
- reference genome
- HEARTY scripts
- ANGSD
- R and Python environments (check HEARTY repo)

## Workflow

1. Run HEARTY ↓
2. Plot minor allele balance / frequency outputs ↓
3. Parse HEARTY het window files and calculate pooled thresholds ↓
4. Generate run_roh_sensitivity.sh ↓
5. Submit/run run_roh_sensitivity.sh ↓
6. Summarise ROH sensitivity outputs into ROH_summary_all.tsv ↓
7. Plot ROH/FROH sensitivity from ROH_summary_all.tsv

## Parameters I used for this workflow (ADJUST IF NEEDED)

- minimum depth: 4
- maximum depth: 50
- HEARTY minor allele balance thresholds: 0.15, 0.20, 0.25, 0.30
- region: scaffold_1
- ROH thresholds: 0.0001, 0.0005, 0.001

## 1. Running HEARTY python script

First we need to actually run the HEARTY python script with the parameters we have set earlier.

▶ HEARTY.py wrapper

```
#!/bin/bash
#SBATCH --job-name=HEARTY_scaf1
#SBATCH --time=03:00:00
```

```
#SBATCH --cpus-per-task=4
#SBATCH --qos=normal
#SBATCH --partition=cpuqueue
#SBATCH --array=1-35%10
#SBATCH --output=jobs/%x_%A_%a.out
#SBATCH --error=jobs/%x_%A_%a.err
#SBATCH --mail-type=END,FAIL
#SBATCH --mail-user=amb230@kent.ac.uk

# Script to run HEARTY to test allele balances per sample


module purge
module load perl/5.38.0 gsl/2.5 bcftools/1.21 python

DIR="/projects/erode/people/qhk515/01_OBP/popgen_analyses/het/HEARTY"
cd $DIR

BAM_LIST_COMBINED="/projects/erode/people/qhk515/01_OBP/scripts/OBPs/list_
OBPs_combined.txt"
REF="/projects/erode/people/qhk515/reference_genomes/OBP/bNeochry1.pri.202
30906.fa"
#LUKE_LIST_SCAF1="/projects/erode/people/qhk515/01_OBP/popgen_analyses/het
/tests_scaf1/scaf1_callable_sites.txt.tab"

# Get sample name from BAM list based on SLURM_ARRAY_TASK_ID
BAM=$(sed -n "${SLURM_ARRAY_TASK_ID}p" ${BAM_LIST_COMBINED})
if [ -z "${BAM}" ]; then
    echo "No BAM file found for SLURM_ARRAY_TASK_ID
${SLURM_ARRAY_TASK_ID}"
    exit 1
fi

SAMPLE=$(basename "${BAM}" .rmdup.bam)

echo "Processing sample ${SAMPLE} with BAM file ${BAM}"

# Apply HEARTY

#All scaffold 1
MAXDEPTH="50"
MINDEPTH="4"

python3 /projects/erode/people/qhk515/software/HEARTY/HEARTY.py --bam
${BAM} \
    -o ${SAMPLE}.allscaf1 --cores ${SLURM_CPUS_PER_TASK} \
    --regions "scaffold_1" --angsd
/opt/shared_software/shared_envmodules/conda/angsd-0.940/bin/angsd --force
\
    --threshold 0.15 \
    --threshold 0.20 \
    --threshold 0.25 \
    --threshold 0.30 \
    --min-depth ${MINDEPTH} \
```

```
    --max-depth ${MAXDEPTH} \
    --reuse-existing
echo "Finished running HEARTY for $SAMPLE "

exit 0
```

## 2. Explore minor allele balance outputs

Now we want to run the scripts that calculate the frequencies of each kind of mutation. This creates individual plots:

▶ Plot_MinorFreq.R wrapper

```
# Later I can test different depth filters because default is 10x
# I tend to run this in a screen session because it's quite quick
module purge

module load miniconda
conda activate r_env

DIR="/projects/erode/people/qhk515/01_OBP/popgen_analyses/het/HEARTY/resul
ts"
cd $DIR

mkdir -p plots_freq
cd plots_freq

for FILE in ${DIR}/*.minorfreq.txt; do
    NAME=$(basename "${FILE}" ".minorfreq.txt")

    Rscript
/projects/erode/people/qhk515/software/HEARTY/Plot_MinorFreq.R \
    --minorfreq-file ${FILE} \
    --out ${NAME} \
    --format pdf
done
```

After running, the script you can examine the individual plots.

Or you can create new plots in R:

▶ R script

```


library(readr)
library(dplyr)
library(tidyr)
library(stringr)
library(purrr)
```

```r
library(ggplot2)
library(patchwork)


# ------------------------------------------------------------
# 1. Directory containing HEARTY minorfreq files
# ------------------------------------------------------------
indir <- "/Users/alexandrabrumwell/OneDrive - University of Kent/PhD
stuff/PhD research/Research projects/01_OBP/Analyses/Het/HEARTY/minorfreq"
dir.create(indir, showWarnings = FALSE)

setwd(indir)

plot_dir <- "plots"

dir.create(plot_dir, showWarnings = FALSE)


# ------------------------------------------------------------
# 2. Find raw minorfreq files
#    Adjust pattern if needed
# ------------------------------------------------------------
files <- list.files(
  path = indir,
  pattern = "minorfreq\\.txt$",
  full.names = TRUE
)

# Exclude summary files
files <- files[!str_detect(files, "summary")]

files

# ------------------------------------------------------------
# 3. Read and combine all individuals
# ------------------------------------------------------------

pair_types <- tibble(
  pair = c("AC", "AG", "AT", "CG", "CT", "GT"),
  mutation_type = c(
    "Transversion",
    "Transition",
    "Transversion",
    "Transversion",
    "Transition",
    "Transversion"
  )
)

minor_df <- map_dfr(files, function(f) {

  sample_name <- basename(f) %>%
    str_remove("\\.allscaf1\\.mincov4.*minorfreq\\.txt$")
```

```
    read_tsv(f, show_col_types = FALSE) %>%
      pivot_longer(
        cols = c(AC, AG, AT, CG, CT, GT),
        names_to = "pair",
        values_to = "count"
      ) %>%
      left_join(pair_types, by = "pair") %>%
      mutate(
        sample = sample_name,
        file = basename(f)
      )
  })

  minor_df <- minor_df %>%
    mutate(
      pair = factor(pair, levels = c("AC", "AG", "AT", "CG", "CT", "GT")),
      mutation_type = factor(mutation_type, levels = c("Transition",
  "Transversion"))
    ) %>%
    mutate(
      Group = if_else(
        str_starts(sample, "O-"),
        "Modern",
        "Historical"
      )
    )

  str(minor_df)


  # Add coverage
  coverage_df <- tribble(
    ~sample, ~MeanCoverage,
    "O-76339-001", 7.18,
    "O-76349-001", 7.43,
    "O-76350-001", 6.95,
    "O-76521-001", 7.04,
    "O-76523-001", 7.08,
    "O-76526-001", 7.07,
    "O-76538-001", 7.07,
    "O-81171-001", 7.10,
    "O-81175-001", 7.09,
    "O-81179-001", 7.26,
    "O-81182-001", 6.97,
    "O-81186-001", 6.84,
    "O-81188-001", 7.01,
    "O-81192-001", 7.16,
    "O-81193-001", 7.10,
    "O-81196-001", 6.95,
    "O-81198-001", 6.08,
    "O-81201-001", 7.01,
    "O-81203-001", 6.93,
    "OBPI_501", 24.06,
    "OBPI_502", 9.97,
```

```
  "OBPI_503", 7.24,
  "OBPI_504", 17.67,
  "OBPI_505", 2.42,
  "OBPI_506", 4.86,
  "OBPI_507", 5.77,
  "OBPI_508", 3.91,
  "OBPII_ReHoI_501", 5.10,
  "OBPII_ReHoI_502", 6.71,
  "OBPII_ReHoI_503", 3.94,
  "OBPII_ReHoI_504", 7.60,
  "OBPII_ReHoI_505", 2.64,
  "OBPII_ReHoI_506", 4.03,
  "OBPII_ReHoI_507", 9.06,
  "OBPII_ReHoI_508", 10.24
)

# Join coverage
minor_df <- minor_df %>%
  left_join(coverage_df, by = "sample")


#Coverage labels for plots
minor_df <- minor_df %>%
  mutate(sample_cov = paste0(sample, " (", MeanCoverage, "x)"))
# Reorder samples
minor_df <- minor_df %>%
  mutate(sample_cov = forcats::fct_reorder(sample_cov, MeanCoverage))

# ------------------------------------------------------------
# PLOT 1: one panel per individual, shared y axis
# ------------------------------------------------------------

p <- ggplot(
  data = minor_df %>%
    filter(Group == "Historical" | sample ==  "O-76339-001"),
  #data = minor_df %>% filter(sample ==  "O-76339-001"),
  aes(
    x = freq,
    y = count,
    colour = pair,
    shape = mutation_type
  )
) +
  geom_point(size = 0.9, alpha = 0.8) +
  geom_line(aes(group = pair), linewidth = 0.5, alpha = 0.7) +
  facet_wrap(Group ~ sample_cov, scales = "fixed", ncol = 4) +
  theme_bw() +
  labs(
    x = "Allele balance / minor allele frequency",
    y = "Number of sites",
    colour = "Base change",
    shape = "Mutation type",
    title = "Minor allele balance distribution across historical samples
OBP individuals (and one modern individual)",
```

```
    subtitle = "Samples ordered by increasing coverage.  > 4x depth
threshold was applied for all sites."
  ) +
  theme(
    legend.position = "bottom",
    strip.text = element_text(size = 6),
    axis.text.x = element_text(angle = 45, hjust = 1),
    # Smaller text
    legend.text = element_text(size = 8),
    legend.title = element_text(size = 9),

  )
p
ggsave(
  filename = file.path(plot_dir,
"01_OBP_minorfreq_all_individuals_shared_y.pdf"),
  plot = p,
  width = 12,
  height = 8
)


# ------------------------------------------------------------
# PLOT 2: Cumulative thresholds
# ------------------------------------------------------------
cdf_pair_df <- minor_df %>%
  group_by(sample, Group, sample_cov, pair, mutation_type, freq) %>%
  summarise(
    total_sites = sum(count, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  group_by(sample, Group, sample_cov, pair, mutation_type) %>%
  arrange(freq, .by_group = TRUE) %>%
  mutate(
    cum_sites = cumsum(total_sites),
    cum_prop_sites = cum_sites / sum(total_sites)
  ) %>%
  ungroup()

p_cdf_pair <- ggplot(
  data = cdf_pair_df %>%
    filter(Group == "Historical" | sample == "O-76339-001") %>%
    filter(sample != "OBPI_504"),
  aes(x = freq, y = cum_prop_sites, colour = pair, linetype =
mutation_type)
) +
  geom_line(linewidth = 0.6, alpha = 0.8) +
  geom_vline(
    xintercept = c(0.05, 0.10, 0.20, 0.25, 0.30),
    linetype = "dashed",
    alpha = 0.35
  ) +
  facet_wrap(~ sample_cov, scales = "fixed", ncol = 4) +
  theme_bw() +
```

```
  labs(
    x = "Allele balance / minor allele frequency threshold",
    y = "Cumulative proportion of sites - only historical samples",
    colour = "Base change",
    linetype = "Mutation type",
    title = "Cumulative allele balance distribution by mutation class",
    subtitle = "Each line shows the cumulative proportion of sites for one
base-change class"
  ) +
  theme(
    legend.position = "bottom",
    strip.text = element_text(size = 6),
    axis.text.x = element_text(angle = 45, hjust = 1)
  )

p_cdf_pair

ggsave(
  filename = file.path(plot_dir,
"02_OBP_cumulative_thresholds_by_mutation_pair.pdf"),
  plot = p_cdf_pair,
  width = 12,
  height = 8
)


# PLOT 3: TRANSITION/ TRANSVERSION RATIO
titv_df <- minor_df %>%

  group_by(Group, sample, freq, mutation_type) %>%

  summarise(count = sum(count), .groups = "drop") %>%

  pivot_wider(
    names_from = mutation_type,
    values_from = count
  ) %>%

  mutate(
    TiTv = Transition / Transversion
  )

p3 <- ggplot(
  titv_df,
  aes(x = freq, y = TiTv, group = sample, colour = Group)
) +
  geom_line(alpha = 0.25) +
  stat_summary(
    aes(group = Group),
    fun = mean,
    geom = "line",
    linewidth = 1.5
  ) +
  theme_bw() +
```

```
  labs(
    x = "Allele balance",
    y = "Transition / Transversion ratio",
    title = "Ti/Tv ratio across allele balance"
  )
p3
ggsave(file.path(plot_dir, "03_TSTV_hist_mod.pdf"), p3, width = 10, height
= 8)
```

## 3. Calculate adaptive ROH thresholds

This workflow has three conceptual steps:

> 1. Generate heterozygosity windows using different heterozygote thresholds
> 2. Define ROHs relative to the heterozygosity distribution
> 3. Compare how ROH/FROH changes across parameter combinations

> Importantly, this analysis uses an alpha value which scales the ROH thresholds (i.e. how many het windows are allowed in a ROH) relative to the data itself because baseline heterozygoisty can vary across inividuals. So using a singular threshold would not be correct or compareable across indivudals. The higher the alpha, the higher the ROH threshold, i.e. the more lenient the threshold is.

### 3A. Generate heterozygosity windows using different heterozygote thresholds

▶ Create wrapper for ROH_Plot_Tool.R

Now we can run a script that calculates het per window and ROHs.

This is based on Hernans script and the script from the HEARTY repository:

```
R
library(data.table)
library(stringr)
library(ggplot2)

# Run this on HPC
DIR <-
"/projects/erode/people/qhk515/01_OBP/popgen_analyses/het/HEARTY/results/R
OH"
dir.create(DIR, showWarnings = FALSE)

setwd(DIR)

PLOT_DIR <- "plots"
dir.create(PLOT_DIR, showWarnings = FALSE)


files <- list.files(
  path =
"/projects/erode/people/qhk515/01_OBP/popgen_analyses/het/HEARTY/results/"
```

```
,
  pattern = "windows(_trv)?\\.txt\\.gz$",
  full.names = TRUE
)
files

parse_hearty_windows <- function(f) {

  lines <- readLines(gzfile(f), warn = FALSE)

  m <- str_match(
    lines,
    "^(\\S+)\\s+(\\d+)-(\\d+):\\s+(\\d+)\\s+sites,\\s+
(\\d+)\\s+sites\\s+with\\s+'HET',\\s+Proportion:\\s+([0-9.eE+-]+)"
  )

  x <- data.table(
    chrom = m[, 2],
    start = as.integer(m[, 3]),
    end = as.integer(m[, 4]),
    total_sites = as.integer(m[, 5]),
    het_sites = as.integer(m[, 6]),
    prop = as.numeric(m[, 7])
  )

  x <- x[!is.na(chrom)]

  base <- basename(f)

  x[, file := f]
  x[, file_id := sub("\\.txt\\.gz$", "", base)]
  x[, sampleID := str_split_fixed(base, "_het", 2)[, 1]]
  x[, hetTsh := as.numeric(str_match(base, "_het([0-9.]+)\\.mincov")[,
2])]
  x[, site_set := ifelse(grepl("_trv\\.txt\\.gz$", base), "trv", "all")]
  x[, window_bp := end - start + 1]
  x[, callable_fraction := total_sites / window_bp]

  return(x)
}

W <- rbindlist(lapply(files, parse_hearty_windows), fill = TRUE)

fwrite(W, paste0(DIR,"/","HEARTY_all_windows_parsed.tsv.gz"), sep = "\t")

W

W[, prop := as.numeric(prop)]
W[, total_sites := as.numeric(total_sites)]
W[, het_sites := as.numeric(het_sites)]
W[, hetTsh := as.numeric(hetTsh)]

S <- W[, .(
  n_windows = as.integer(.N),
```

```
  mean_het = as.numeric(mean(prop, na.rm = TRUE)),
  median_het = as.numeric(median(prop, na.rm = TRUE)),
  q001 = as.numeric(quantile(prop, 0.001, na.rm = TRUE)),
  q005 = as.numeric(quantile(prop, 0.005, na.rm = TRUE)),
  q01  = as.numeric(quantile(prop, 0.01,  na.rm = TRUE)),
  q025 = as.numeric(quantile(prop, 0.025, na.rm = TRUE)),
  q05  = as.numeric(quantile(prop, 0.05,  na.rm = TRUE)),
  q10  = as.numeric(quantile(prop, 0.10,  na.rm = TRUE)),
  median_sites = as.numeric(median(total_sites, na.rm = TRUE)),
  mean_sites = as.numeric(mean(total_sites, na.rm = TRUE))
), by = .(file, file_id, sampleID, hetTsh, site_set)]

fwrite(S, paste0(DIR,"/","HEARTY_window_het_summary_by_file.tsv"), sep =
"\t")

S[order(site_set, hetTsh, sampleID)]

W[, log_prop := log10(prop + 1e-8)]

# Plotting
P <- ggplot(W, aes(x = log_prop, colour = sampleID)) +
  geom_density(linewidth = 0.4) +
  facet_grid(site_set ~ hetTsh, scales = "free_y") +
  theme_bw() +
  labs(
    x = "log10(window heterozygosity + 1e-8)",
    y = "Density",
    title = "Window heterozygosity distributions",
  ) +
  theme(
    legend.position = "bottom",

    # Smaller text
    legend.text = element_text(size = 6),
    legend.title = element_text(size = 7),

    # Smaller spacing/keys
    legend.key.size = unit(0.3, "cm"),
    legend.spacing.x = unit(0.1, "cm"),

    # Optional
    legend.margin = margin(0, 0, 0, 0)
  )

ggsave(paste0(PLOT_DIR,"/","HEARTY_window_heterozygosity_density.png"), P,
width = 14, height = 8)

# Create alpha values
alpha_grid <- c(0.10, 0.20, 0.30, 0.50)

Adaptive <- S[, .(alpha = alpha_grid),
              by = .(file, file_id, sampleID, hetTsh, site_set,
median_het)]
```

```
Adaptive[, roh_threshold := median_het * alpha]
Adaptive[, threshold_type := paste0("file_median_x", alpha)]

fwrite(Adaptive,
paste0(DIR,"/","HEARTY_adaptive_roh_thresholds_by_file.tsv"), sep = "\t")

Adaptive[order(site_set, hetTsh, sampleID, alpha)]

Pooled <- W[, .(
  pooled_mean_het = mean(prop, na.rm = TRUE),
  pooled_median_het = median(prop, na.rm = TRUE),
  pooled_q01 = quantile(prop, 0.01, na.rm = TRUE),
  pooled_q05 = quantile(prop, 0.05, na.rm = TRUE),
  pooled_q10 = quantile(prop, 0.10, na.rm = TRUE)
), by = .(hetTsh, site_set)]

Pooled_alpha <- Pooled[, .(alpha = alpha_grid),
                       by = .(hetTsh, site_set, pooled_median_het)]

Pooled_alpha[, roh_threshold := pooled_median_het * alpha]
Pooled_alpha[, threshold_type := paste0("pooled_median_x", alpha)]

fwrite(Pooled_alpha,
paste0(DIR,"/","HEARTY_pooled_roh_thresholds_by_class.tsv"), sep = "\t")

Pooled_alpha[order(site_set, hetTsh, alpha)]
names(Pooled_alpha)


ROH_Plot_Tool <-
"/projects/erode/people/qhk515/software/HEARTY/ROH_Plot_Tool.R"


# Check available values
sort(unique(Pooled_alpha$hetTsh))
sort(unique(Pooled_alpha$alpha))

# Choose which HEARTY heterozygosity thresholds to include
use_het <- c(0.15, 0.20, 0.25, 0.30)

# Choose which ROH-threshold multipliers to include
use_alpha <- c(0.20, 0.30, 0.50)

# Filter pooled threshold table
thresholds_to_use <- Pooled_alpha[
  hetTsh %in% use_het & alpha %in% use_alpha
]

thresholds_to_use

file_info <- unique(S[, .(file, file_id, sampleID, hetTsh, site_set)])

Jobs <- merge(
  file_info,
```

```
  thresholds_to_use,
  by = c("hetTsh", "site_set"),
  allow.cartesian = TRUE
)

Jobs[, out_prefix := paste0(
  "roh_sensitivity/",
  file_id,
  "_",
  threshold_type,
  "_roh",
  signif(roh_threshold, 3)
)]

Jobs[, pdf := paste0(out_prefix, ".pdf")]
Jobs[, summary := paste0(out_prefix, "_summary.txt")]

Jobs[, cmd := sprintf(
  paste(
    'Rscript "%s"',
    '--mode single',
    '--input "%s"',
    '--output-pdf "%s"',
    '--min-length-mb 10',
    '--max-scaffolds 40',
    '--fill-cap auto',
    '--fill-cap-scale 0.8',
    '--bin-size-bp 500000',
    '--fallback-resolution 100',
    '--roh-threshold %.8g',
    '2>&1 | tee "%s"'
  ),
  ROH_Plot_Tool,
  file,
  pdf,
  roh_threshold,
  summary
)]

writeLines(
  c("#!/bin/bash", "mkdir -p roh_sensitivity", Jobs$cmd),
  "run_roh_sensitivity.sh"
)

fwrite(Jobs, paste0(DIR,"/","ROH_sensitivity_jobs.tsv"), sep = "\t")
q()
```

### 3B. Run ROH script

Now you can submit this script in a screen session: `bash run_roh_sensitivity.sh`

▶ Optional: exploratory fixed-threshold ROH run

```
# ROH PLOTS
module purge
module load miniconda
conda activate r_env

ROH_Plot_Tool="/projects/erode/people/qhk515/software/HEARTY/ROH_Plot_Tool
.R"

DIR="/projects/erode/people/qhk515/01_OBP/popgen_analyses/het/HEARTY/resul
ts/ROH/"
cd $DIR

for FILE in ${DIR}/O-76339-001*window*gz; do
    for THR in 0.0001 0.0005 0.001; do


    OUT=$(basename ${FILE} .txt.gz)
    PDF="${OUT}_roh_${THR}.pdf"
    SUMMARY="${OUT}_roh_${THR}_summary.txt"

    echo "Running:"
    echo "Input:   ${FILE}"
    echo "PDF:     ${PDF}"
    echo "Summary: ${SUMMARY}"

    Rscript "${ROH_Plot_Tool}" \
      --mode single \
      --input "${FILE}" \
      --output-pdf "${PDF}" \
      --min-length-mb 10   \
      --max-scaffolds 40 \
      --fill-cap auto --fill-cap-scale 0.8 \
      --bin-size-bp 500000 \
      --fallback-resolution 100 \
      --roh-threshold "${THR}" 2>&1 | tee "${SUMMARY}"

  done
done
```

### 3C. Summarise ROH outputs

▶ Summary script

```
#!/bin/bash

DIR="/projects/erode/people/qhk515/01_OBP/popgen_analyses/het/HEARTY/resul
ts/ROH/roh_sensitivity"
cd $DIR

OUT="ROH_summary_all.tsv"
```

```
echo -e
"file\tsample\tthreshold\tmean_het_all\tmedian_het_all\trows_plotted\tmean
_het_plotted\tmedian_het_plotted\traw_roh_bins\tbridged_roh_bins\tgenome_s
ize_bp\tn_windows\twindow_size_bp\tFROH_raw_100kb\tROH_bp_raw_100kb\tFROH_
raw_1Mb\tROH_bp_raw_1Mb\tFROH_raw_5Mb\tROH_bp_raw_5Mb\tFROH_bridged_100kb\
tROH_bp_bridged_100kb\tFROH_bridged_1Mb\tROH_bp_bridged_1Mb\tFROH_bridged_
5Mb\tROH_bp_bridged_5Mb" > "${OUT}"

for f in *_summary.txt; do

  awk -v file="$f" '
  BEGIN {
    mean_all="NA"; median_all="NA";
    rows_plotted="NA"; mean_plotted="NA"; median_plotted="NA";
    raw_bins="NA"; bridged_bins="NA";
    genome_bp="NA"; n_windows="NA"; window_bp="NA";

    froh_raw_100="NA"; rohbp_raw_100="NA";
    froh_raw_1="NA"; rohbp_raw_1="NA";
    froh_raw_5="NA"; rohbp_raw_5="NA";

    froh_bridged_100="NA"; rohbp_bridged_100="NA";
    froh_bridged_1="NA"; rohbp_bridged_1="NA";
    froh_bridged_5="NA"; rohbp_bridged_5="NA";

    section="";
  }

  /^Mean heterozygosity \(all parsed windows\):/ {
    mean_all=$NF
  }

  /^Median heterozygosity \(all parsed windows\):/ {
    median_all=$NF
  }

  /^Rows used for plotted scaffolds:/ {
    rows_plotted=$NF
  }

  /^Mean heterozygosity \(plotted scaffolds\):/ {
    mean_plotted=$NF
  }

  /^Median heterozygosity \(plotted scaffolds\):/ {
    median_plotted=$NF
  }

  /^Raw ROH bins flagged:/ {
    raw_bins=$NF
  }

  /^Bridged ROH bins flagged:/ {
```

```
    bridged_bins=$NF
  }

  /^Genome size used for FROH:/ {
    genome_bp=$6

    tmp=$0
    sub(/^.*n_windows=/, "", tmp)
    sub(/ .*/, "", tmp)
    n_windows=tmp

    tmp2=$0
    sub(/^.*x /, "", tmp2)
    sub(/ bp.*$/, "", tmp2)
    window_bp=tmp2
  }

  /^FROH \(unbridged\/raw/ {
    section="raw"
  }

  /^FROH \(bridged/ {
    section="bridged"
  }

  /^[[:space:]]*>100kb:/ {
    val=$2
    bp=$3
    gsub(/\(ROH/, "", bp)
    gsub(/bp=/, "", bp)
    gsub(/\)/, "", bp)

    if (section=="raw") {
      froh_raw_100=val
      rohbp_raw_100=bp
    } else if (section=="bridged") {
      froh_bridged_100=val
      rohbp_bridged_100=bp
    }
  }

  /^[[:space:]]*>1Mb:/ {
    val=$2
    bp=$3
    gsub(/\(ROH/, "", bp)
    gsub(/bp=/, "", bp)
    gsub(/\)/, "", bp)

    if (section=="raw") {
      froh_raw_1=val
      rohbp_raw_1=bp
    } else if (section=="bridged") {
      froh_bridged_1=val
      rohbp_bridged_1=bp
```

```
    }
  }

  /^[[:space:]]*>5Mb:/ {
    val=$2
    bp=$3
    gsub(/\(ROH/, "", bp)
    gsub(/bp=/, "", bp)
    gsub(/\)/, "", bp)

    if (section=="raw") {
      froh_raw_5=val
      rohbp_raw_5=bp
    } else if (section=="bridged") {
      froh_bridged_5=val
      rohbp_bridged_5=bp
    }
  }

  END {
    base=file
    sub(/^.*\//, "", base)
    sub(/_summary\.txt$/, "", base)

    sample=base
    threshold="NA"

      if (base ~ /_roh[^_]+$/) {

        threshold = base
        sub(/^.*_roh/, "", threshold)

        sample = base
        sub(/_roh[^_]+$/, "", sample)
      }

    print file "\t" sample "\t" threshold "\t" \
          mean_all "\t" median_all "\t" rows_plotted "\t" \
          mean_plotted "\t" median_plotted "\t" \
          raw_bins "\t" bridged_bins "\t" \
          genome_bp "\t" n_windows "\t" window_bp "\t" \
          froh_raw_100 "\t" rohbp_raw_100 "\t" \
          froh_raw_1 "\t" rohbp_raw_1 "\t" \
          froh_raw_5 "\t" rohbp_raw_5 "\t" \
          froh_bridged_100 "\t" rohbp_bridged_100 "\t" \
          froh_bridged_1 "\t" rohbp_bridged_1 "\t" \
          froh_bridged_5 "\t" rohbp_bridged_5
  }
  ' "$f" >> "${OUT}"

done

echo "Wrote: ${OUT}"
```

### 3D. Create plots

Use this summary file to create plots for ROH and het. We can create this using R on the HPC or locally.

▶ R script - ROH plots

```
# ============================================================
# Plot HEARTY ROH sensitivity summaries
# Input: ROH_summary_all.tsv
# ============================================================

library(data.table)
library(stringr)
library(ggplot2)
library(tidyr)
library(dplyr)
library(httpgd)

# If you want to see plots live:
# hgd()
# ----------------------------
# 1. Paths
# ----------------------------

DIR <-
"/projects/erode/people/qhk515/01_OBP/popgen_analyses/het/HEARTY/results/ROH/roh_sensitivity"

#DIR <- "~/Library/CloudStorage/OneDrive-UniversityofKent/PhD stuff/PhD research/Research projects/01_OBP/Analyses/Het/HEARTY/roh"

INFILE <- file.path(DIR, "ROH_summary_all.tsv")
PLOT_DIR <- file.path(DIR, "plots_ROH_summary")

dir.create(PLOT_DIR, showWarnings = FALSE, recursive = TRUE)

# ----------------------------
# 2. Read summary file
# ----------------------------

ROH <- fread(INFILE, na.strings = c("NA", "", "NaN"))

# Convert numeric columns
num_cols <- setdiff(names(ROH), c("file", "sample"))

ROH[, (num_cols) := lapply(.SD, as.numeric), .SDcols = num_cols]

# ----------------------------
# 3. Parse sample / thresholds
# ----------------------------
```

```
ROH[, sample_base := sample]


# Sample ID = everything before ".allscaf1_het"
ROH[, sampleID := str_match(sample_base, "^(.+?)\\.allscaf1_het")[, 2]]

# HEARTY heterozygote threshold
ROH[, hetTsh := as.numeric(str_match(sample_base, "_het([0-
9.]+)\\.mincov")[, 2])]

# Site set: transversions-only or all sites
ROH[, site_set := fifelse(str_detect(sample_base, "_trv|\\.trv|trv"),
"trv", "all")]

# ROH threshold multiplier alpha
ROH[, alpha := as.numeric(str_match(sample_base, "pooled_median_x([0-
9.]+)")[, 2])]

# Fallback: if alpha is not in sample name, infer from threshold /
median_het_all
ROH[is.na(alpha) & !is.na(threshold) & !is.na(median_het_all),
    alpha := threshold / median_het_all]

# Make labels cleaner
ROH[, hetTsh_f := factor(hetTsh, levels = sort(unique(hetTsh)))]
ROH[, alpha_f := factor(alpha, levels = sort(unique(alpha)))]

# Check parsing
print(ROH[, .N, by = .(sampleID, hetTsh, site_set, alpha)][order(sampleID,
site_set, hetTsh, alpha)])


# Samples I am interested in
coverage_df <- tribble(
  ~sample, ~MeanCoverage,
  "O-76339-001", 7.18,
  "O-76349-001", 7.43,
  "O-76350-001", 6.95,
  "O-76521-001", 7.04,
  "O-76523-001", 7.08,
  "O-76526-001", 7.07,
  "O-76538-001", 7.07,
  "O-81171-001", 7.10,
  "O-81175-001", 7.09,
  "O-81179-001", 7.26,
  "O-81182-001", 6.97,
  "O-81186-001", 6.84,
  "O-81188-001", 7.01,
  "O-81192-001", 7.16,
  "O-81193-001", 7.10,
  "O-81196-001", 6.95,
  "O-81198-001", 6.08,
  "O-81201-001", 7.01,
  "O-81203-001", 6.93,
```

```
  "OBPI_501", 24.06,
  "OBPI_502", 9.97,
  "OBPI_503", 7.24,
  "OBPI_504", 17.67,
  "OBPI_505", 2.42,
  "OBPI_506", 4.86,
  "OBPI_507", 5.77,
  "OBPI_508", 3.91,
  "OBPII_ReHoI_501", 5.10,
  "OBPII_ReHoI_502", 6.71,
  "OBPII_ReHoI_503", 3.94,
  "OBPII_ReHoI_504", 7.60,
  "OBPII_ReHoI_505", 2.64,
  "OBPII_ReHoI_506", 4.03,
  "OBPII_ReHoI_507", 9.06,
  "OBPII_ReHoI_508", 10.24
) %>%
  mutate(
    Group = if_else(str_starts(sample, "O-"),
                    "Modern",
                    "Historical")
  )

# Join dataframes
ROH <- ROH %>%
  left_join(
    coverage_df,
    by = c("sampleID" = "sample")
  )

# ============================================================
# 4. PLOT 1: Mean heterozygosity across HEARTY thresholds
# ============================================================
# Create fixed colour mapping
sample_cols <- setNames(
  scales::hue_pal()(length(unique(het_summary$sampleID))),
  sort(unique(het_summary$sampleID))
)

het_summary <- unique(
  ROH[, .(
    sampleID,
    hetTsh,
    hetTsh_f,
    site_set,
    mean_het_all,
    MeanCoverage,
    Group
  )]
)



p_het <- ggplot(
  het_summary %>% filter(Group == "Historical"),
```

```
  aes(x = hetTsh, y = mean_het_all, colour = sampleID, group = sampleID,
linetype = Group)
) +
  geom_point(size = 2.5) +
  geom_line(linewidth = 0.8) +
  facet_wrap(~ site_set, scales = "free_y") +
  scale_colour_manual(values = sample_cols) +
  theme_bw() +
  labs(
    title = "Mean heterozygosity across HEARTY thresholds - ONLY
HISTORICAL",
    x = "HEARTY heterozygote threshold",
    y = "Mean window heterozygosity",
    colour = "sampleID"
  ) +
  theme(
    strip.text = element_text(size = 6),
    axis.text.x = element_text(angle = 45, hjust = 1),
    # Smaller text
    legend.text = element_text(size = 8),
    legend.title = element_text(size = 9),

  )

print(p_het)
ggsave(
  file.path(PLOT_DIR, "01_mean_heterozygosity_by_hetTsh_historical.png"),
  p_het,
  width = 14,
  height = 7,
  dpi = 300
)

 # ============================================================
# 5. PLOT 2: Sensitivity of FROH raw >1Mb
# ============================================================

p_raw_1Mb <- ggplot(
  ROH,
  aes(x = alpha, y = FROH_raw_1Mb, colour = sampleID, group = sampleID,
linetype = Group)
) +
  geom_point(size = 2.2) +
  geom_line(linewidth = 0.7) +
  facet_grid(site_set + Group ~ hetTsh_f, scales = "fixed") +
  scale_colour_manual(values = sample_cols) +
  theme_bw() +
  labs(
    title = "Sensitivity of FROH >1Mb to ROH threshold",
    x = "ROH threshold multiplier: pooled median × alpha",
    y = "FROH raw >1Mb",
    colour = "sampleID"
  )
print(p_raw_1Mb)
```

```
ggsave(
  file.path(PLOT_DIR, "02_FROH_raw_1Mb_sensitivity.png"),
  p_raw_1Mb,
  width = 14,
  height = 8,
  dpi = 300
)

# ============================================================
# 6. Sensitivity of FROH raw >5Mb
# ============================================================
p_raw_5Mb <- ggplot(
  ROH,
  aes(x = alpha, y = FROH_raw_5Mb, colour = sampleID, group = sampleID,
linetype = Group)
) +
  geom_point(size = 2.2) +
  geom_line(linewidth = 0.7) +
  facet_grid(site_set + Group ~ hetTsh_f, scales = "fixed") +
  scale_colour_manual(values = sample_cols) +
  theme_bw() +
  labs(
    title = "Sensitivity of FROH >5Mb to ROH threshold",
    x = "ROH threshold multiplier: pooled median × alpha",
    y = "FROH raw >5Mb",
    colour = "sampleID"
  )
print(p_raw_5Mb)
ggsave(
  file.path(PLOT_DIR, "03_FROH_raw_5Mb_sensitivity.png"),
  p_raw_5Mb,
  width = 14,
  height = 8,
  dpi = 300
)

# ============================================================
# 7. Raw versus bridged FROH
# ============================================================

ROH_long <- ROH %>%
  as_tibble() %>%
  select(
    sampleID, hetTsh, hetTsh_f, site_set, alpha,
    FROH_raw_1Mb, FROH_bridged_1Mb,
    FROH_raw_5Mb, FROH_bridged_5Mb
  ) %>%
  pivot_longer(
    cols = starts_with("FROH_"),
    names_to = c("bridge", "length_class"),
    names_pattern = "FROH_(raw|bridged)_(1Mb|5Mb)",
    values_to = "FROH"
  ) %>%
```

```
  mutate(
    bridge = factor(bridge, levels = c("bridged", "raw")),
    length_class = factor(length_class, levels = c("1Mb", "5Mb"),
                          labels = c(">1Mb", ">5Mb"))
  )

p_raw_bridged <- ggplot(
  ROH_long,
  aes(
    x = alpha,
    y = FROH,
    colour = sampleID,
    group = interaction(sampleID, bridge),
    linetype = bridge
  )
) +
  geom_point(size = 1.8) +
  geom_line(linewidth = 0.7) +
  facet_grid(length_class + site_set ~ hetTsh_f) +
  scale_colour_manual(values = sample_cols) +
  theme_bw() +
  labs(
    title = "Raw versus bridged FROH",
    x = "ROH threshold multiplier: pooled median × alpha",
    y = "FROH",
    colour = "sampleID",
    linetype = "bridge"
  )
print(p_raw_bridged)
ggsave(
  file.path(PLOT_DIR, "04_FROH_raw_vs_bridged.png"),
  p_raw_bridged,
  width = 16,
  height = 12,
  dpi = 300
)

# ============================================================
# 8. All-sites versus transversions-only FROH raw >1Mb
# ============================================================

wide_1Mb <- ROH %>%
  as_tibble() %>%
  select(sampleID, hetTsh, hetTsh_f, alpha, alpha_f, site_set,
FROH_raw_1Mb, MeanCoverage, Group) %>%
  pivot_wider(
    names_from = site_set,
    values_from = FROH_raw_1Mb
  ) %>%
  filter(!is.na(all), !is.na(trv))

p_all_trv_1Mb <- ggplot(
  wide_1Mb,
  aes(x = all, y = trv, colour = sampleID)
```

```
  ) +
    geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour =
"black") +
    geom_point(size = 2.4) +
    facet_grid(alpha_f ~ hetTsh_f) +
    scale_colour_manual(values = sample_cols) +
    geom_text_repel(
      aes(label = sampleID),
      size = 2.5,
      max.overlaps = 15,
      show.legend = FALSE,
      # Push labels further away
      force = 10,
      force_pull = 0.5,
      # More spacing
      box.padding = 0.8,
      point.padding = 0.8,
      # Segment line
      segment.alpha = 0.5,
      segment.size = 0.3
    ) +
    theme_bw() +
    labs(
      title = "All-sites versus transversions-only FROH",
      x = "All-sites FROH raw >1Mb",
      y = "Transversions-only FROH raw >1Mb",
      colour = "sampleID"
    )
  print(p_all_trv_1Mb)
  ggsave(
    file.path(PLOT_DIR, "05_all_vs_trv_FROH_raw_1Mb.png"),
    p_all_trv_1Mb,
    width = 18,
    height = 6,
    dpi = 300
  )

  # ============================================================
  # 9. All-sites versus transversions-only FROH raw >5Mb
  # ============================================================

  wide_5Mb <- ROH %>%
    as_tibble() %>%
    select(sampleID, hetTsh, hetTsh_f, alpha, alpha_f, site_set,
FROH_raw_5Mb) %>%
    pivot_wider(
      names_from = site_set,
      values_from = FROH_raw_5Mb
    ) %>%
    filter(!is.na(all), !is.na(trv))

  p_all_trv_5Mb <- ggplot(
    wide_5Mb,
    aes(x = all, y = trv, colour = sampleID)
```

```
  ) +
    geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour =
"black") +
    geom_point(size = 2.4) +
    facet_grid(alpha_f ~ hetTsh_f) +
    scale_colour_manual(values = sample_cols) +
    geom_text_repel(
      aes(label = sampleID),
      size = 2.5,
      max.overlaps = 15,
      show.legend = FALSE,
      # Push labels further away
      force = 10,
      force_pull = 0.5,
      # More spacing
      box.padding = 0.8,
      point.padding = 0.8,
      # Segment line
      segment.alpha = 0.5,
      segment.size = 0.3
    ) +
    theme_bw() +
    labs(
      title = "All-sites versus transversions-only long ROH",
      x = "All-sites FROH raw >5Mb",
      y = "Transversions-only FROH raw >5Mb",
      colour = "sampleID"
    )
  print(p_all_trv_5Mb)
  ggsave(
    file.path(PLOT_DIR, "06_all_vs_trv_FROH_raw_5Mb.png"),
    p_all_trv_5Mb,
    width = 18,
    height = 6,
    dpi = 300
  )

  # ============================================================
  # 10. Sensitivity range across ROH thresholds
  # ============================================================

  range_df <- ROH %>%
    as_tibble() %>%
    group_by(sampleID, site_set, Group) %>%
    summarise(
      min_FROH = min(FROH_raw_1Mb, na.rm = TRUE),
      mean_FROH = mean(FROH_raw_1Mb, na.rm = TRUE),
      max_FROH = max(FROH_raw_1Mb, na.rm = TRUE),
      .groups = "drop"
    )

  sample_order <- range_df %>%
    group_by(sampleID) %>%
    summarise(mean_order = mean(mean_FROH, na.rm = TRUE), .groups = "drop")
```

```
  %>%
    arrange(mean_order) %>%
    pull(sampleID)

  range_df <- range_df %>%
    mutate(sampleID = factor(sampleID, levels = sample_order))

  p_range <- ggplot(
    range_df,
    aes(y = sampleID, colour = site_set)
  ) +
    geom_segment(
      aes(x = min_FROH, xend = max_FROH, yend = sampleID),
      linewidth = 0.8
    ) +
    geom_point(aes(x = mean_FROH), size = 3) +
    theme_bw() +
    labs(
      title = "Sensitivity range across ROH thresholds",
      x = "FROH raw >1Mb, mean and range across settings",
      y = "Sample",
      colour = "site_set"
    )
  print(p_range)
  ggsave(
    file.path(PLOT_DIR, "07_FROH_sensitivity_range_by_sample.png"),
    p_range,
    width = 12,
    height = 8,
    dpi = 300
  )




  # PLOT 1: COV VS HET BY THRESHOLD
  p_cov_het <- ggplot(
    het_summary,
    aes(x = MeanCoverage, y = mean_het_all, colour = Group)
  ) +
    geom_point(size = 2.5, alpha = 0.8) +
    geom_smooth(method = "lm", se = FALSE) +
    facet_grid(site_set ~ hetTsh_f, scales = "free_y") +
    theme_bw() +
    labs(
      title = "Effect of coverage on mean window heterozygosity",
      x = "Mean coverage",
      y = "Mean window heterozygosity",
      colour = "Group"
    )

  print(p_cov_het)

  ggsave(
```

```
  file.path(PLOT_DIR, "08_coverage_vs_mean_heterozygosity.png"),
  p_cov_het,
  width = 14,
  height = 8,
  dpi = 300
)



# ----------------------------
# 11. Save parsed table
# ----------------------------

fwrite(
  ROH,
  file.path(PLOT_DIR, "ROH_summary_all_parsed_for_plotting.tsv"),
  sep = "\t"
)

message("Done. Plots written to: ", PLOT_DIR)
```